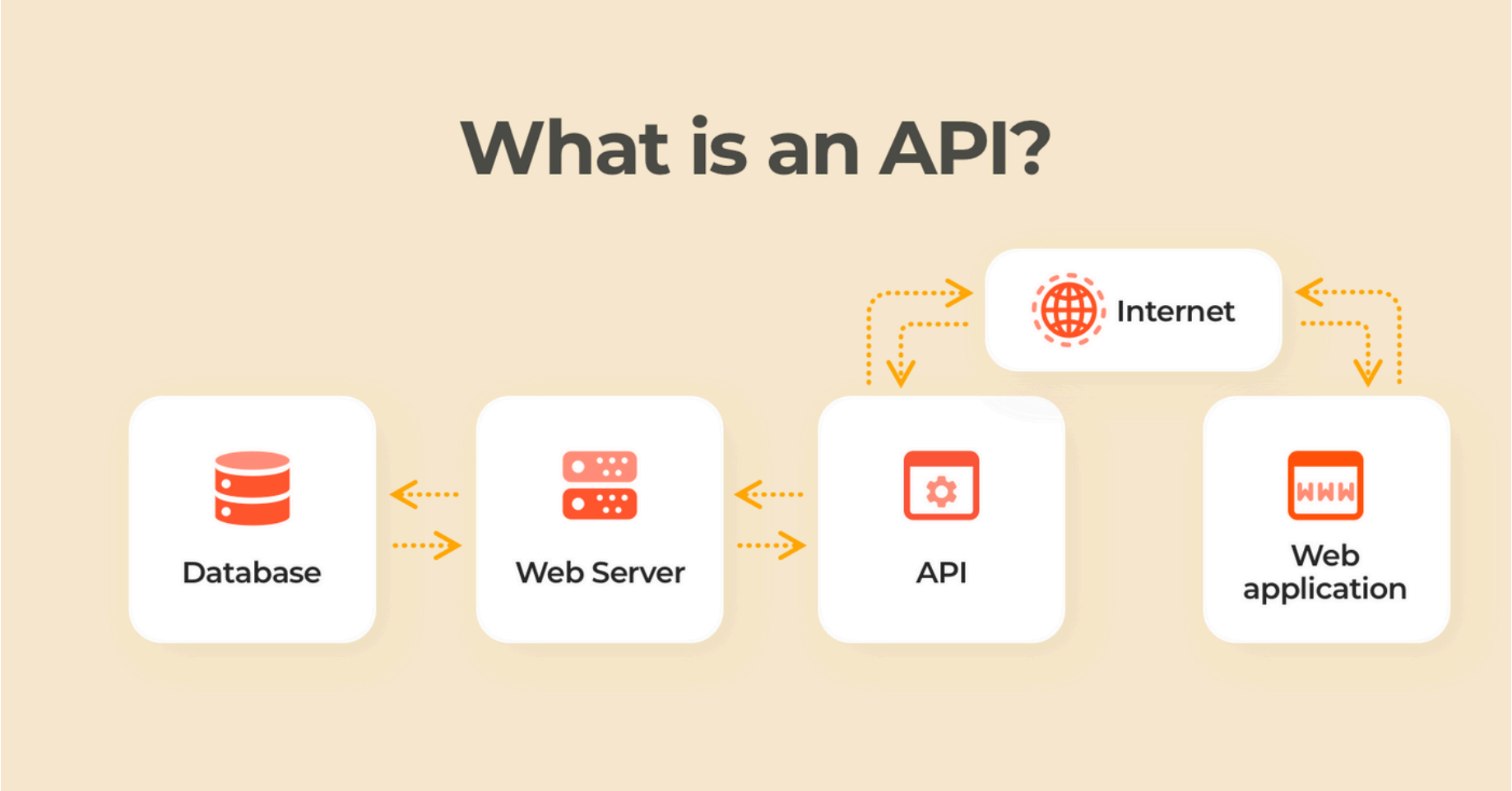
1.API – Introduction

What is an API ?

- API stands for Application Programming Interface
- An API is a contract between two pieces of software that defines how they can talk to each other
- It defines:
  - What requests can be made
  - How to make those requests
  - What response to expect back
- The word Interface is the key — it is a defined boundary through which communication happens
- You do not need to know what is happening inside — you just need to know the interface



The real world analogy

- Think of a restaurant
- You are the client, the kitchen is the server
- You do not walk into the kitchen and cook yourself
- You interact through a menu and a waiter
- The menu is the API — it defines exactly what you can order, in what format, and what you will get back
- You say "I want item 4" — you do not care how the kitchen prepares it
- The kitchen returns a plate of food — the agreed response
- The waiter is the communication channel
- The contract is — if you order item 4, you get pasta. Every time. That is the API guarantee.

What an API Does

- Exposes functionality of a system to the outside world in a controlled way
- Abstracts the internal implementation — callers do not see or care about the code inside
- Receives a request in a defined format
- Processes it internally (database queries, business logic, computations)
- Returns a response in a defined format
- Enforces rules — authentication, rate limiting, input validation — before processing

Concrete example

- You open a weather app on your phone
- The app calls a weather API: GET /weather?city=Bengaluru
- The weather service receives this, queries its internal database and forecasting models

- Returns: temperature, humidity, forecast — in JSON
- Your app displays it
- Your app has no idea how the weather service works internally
- It just knows the contract — send this request, get this response

---

Why APIs Exist

- This is the more important question — understanding the why gives you the system design intuition

Abstraction and separation of concerns

- Without APIs, every system would need to know the internals of every other system it talks to
- APIs create a clean boundary — you expose what you want to expose, hide everything else
- Internal implementation can change completely as long as the API contract stays the same
- Example — Twitter can rewrite its entire backend in a different language, the API stays the same, all apps using it keep working

Enabling interoperability

- Different systems written in different languages on different platforms need to talk to each other
- APIs provide a language-agnostic communication layer
- A Python backend, a React frontend, an iOS app, and an Android app all speak to the same API
- They do not care about each other's internals — just the shared contract

Reusability

- Build something once, expose it as an API, let many clients use it
- Stripe built a payment API once — thousands of companies use it without building payment infrastructure themselves
- Google Maps API — millions of apps use it without running their own mapping infrastructure
- This is the business model of API-first companies

Security and access control

- Without an API layer, you would have to give direct database access to every client
- API acts as a gatekeeper — validates authentication, checks permissions, enforces rate limits
- Only the operations you explicitly expose are accessible
- Everything else in your system is invisible and unreachable

Scalability and independent deployability

- In a monolith, everything is coupled — changing one thing can break everything
- APIs decouple systems — each service exposes an API, others call it
- Services can be scaled, deployed, and updated independently
- This is the foundation of microservices architecture

Monetisation

- APIs are products
- Twilio sells SMS sending as an API
- AWS sells compute, storage, and databases as APIs
- Companies expose APIs to third party developers to build ecosystems around their platform
- Twitter, Facebook, Google — all built developer ecosystems through public APIs

---

Client-Server Communication

- Client-server is the fundamental model that APIs operate on
- One side makes requests, the other side responds

What is a client

- Any system that initiates a request
- Browser, mobile app, another backend service, CLI tool, IoT device
- The client does not have the data or capability it needs — it asks the server for it
- Client is the consumer of the API

What is a server

- Any system that listens for requests and responds
- Holds data, runs business logic, manages resources
- The server is the producer — it has what the client needs
- Server is the one exposing the API

The communication model

- Client sends a request to the server's API endpoint
- Request contains:
  - What it wants (the endpoint and method)
  - Who is asking (authentication)
  - Any data needed to fulfil the request (body or query params)
- Server receives the request, validates it, processes it, returns a response
- Response contains:
  - Whether it succeeded or failed (status code)
  - The requested data or error details (body)
- Connection ends — server forgets about this interaction (stateless)

The same entity can be both client and server

- In microservices, Service A calls Service B's API — A is a client, B is a server
- Service B might itself call Service C's API — now B is also a client
- This is extremely common in distributed systems
- A single user request might trigger a chain of client-server interactions across 5 or 6 internal services

---